

DECISION TREE

```
import pandas as pd
```

```
df = pd.read_csv("Customer_Behaviour.csv")  
print(df.columns)  
print(df.head())
```

```
import pandas as pd  
import matplotlib.pyplot as plt  
from sklearn.tree import DecisionTreeClassifier, plot_tree  
from sklearn.preprocessing import LabelEncoder  
from sklearn.model_selection import train_test_split  
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
# Load dataset
```

```
df = pd.read_csv("Customer_Behaviour.csv")
```

```
# Convert Gender column into numeric values
```

```
le = LabelEncoder()
```

```
df["Gender"] = le.fit_transform(df["Gender"])
```

```
# Features and Target
```

```
X = df[["Gender", "Age", "EstimatedSalary"]]
```

```
y = df["Purchased"]
```

```
# Split dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.3, random_state=42  
)
```

```
# Create Decision Tree model
```

```
model = DecisionTreeClassifier(criterion="entropy")
```

```
# Train model
```

```
model.fit(X_train, y_train)
```

```
# Predict
```

```
y_pred = model.predict(X_test)
```

```
# Accuracy
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
# Classification Report
```

```
print("\nClassification Report:")
```

```

print(classification_report(y_test, y_pred))

# Confusion Matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Plot Decision Tree
plt.figure(figsize=(12, 8))
plot_tree(
    model,
    feature_names=["Gender", "Age", "EstimatedSalary"],
    class_names=["Not Purchased", "Purchased"],
    filled=True
)

plt.show()

```

SVM

```

import pandas as pd
df = pd.read_csv("weather_forecast_data.csv")
print(df.columns)
print(df.head())

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix

# Load dataset
df = pd.read_csv("weather_forecast_data.csv")

# Convert Rain column into numeric values
le = LabelEncoder()
df["Rain"] = le.fit_transform(df["Rain"])

# Features and Target
X = df[["Temperature", "Humidity", "Wind_Speed",
        "Cloud_Cover", "Pressure"]]

```

```

y = df["Rain"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create SVM model
svm = SVC(kernel="linear")

# Train model
svm.fit(X_train, y_train)

# Prediction
y_pred = svm.predict(X_test)

# Accuracy
print("SVM Accuracy:", accuracy_score(y_test, y_pred))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Heatmap
plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d')
plt.title("SVM Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

```

Navie bayes

Uper wala chota code lena hai

```

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.preprocessing import LabelEncoder

```

```

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix

# Load dataset
df = pd.read_csv("StudentsPerformance.csv")

# Create target column (Pass/Fail based on Math Score)
df["Result"] = (df["math score"] >= 40).astype(int)

# Convert categorical columns into numeric
le = LabelEncoder()

df["gender"] = le.fit_transform(df["gender"])
df["race/ethnicity"] = le.fit_transform(df["race/ethnicity"])
df["parental level of education"] = le.fit_transform(df["parental level of education"])
df["lunch"] = le.fit_transform(df["lunch"])
df["test preparation course"] = le.fit_transform(df["test preparation course"])

# Features and Target
X = df.drop("Result", axis=1)
y = df["Result"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create Naive Bayes model
nb = GaussianNB()

# Train model
nb.fit(X_train, y_train)

# Prediction
y_pred = nb.predict(X_test)

# Accuracy
print("Naive Bayes Accuracy:", accuracy_score(y_test, y_pred))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)

```

```
# Heatmap
plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Naive Bayes Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Logistic Regression

```
import pandas as pd
```

```
df = pd.read_csv("StudentsPerformance.csv")
print(df.columns)
print(df.head())
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
# Load Dataset
df = pd.read_csv("Customer_Behaviour.csv")
```

```
# Display first 5 rows
print("Dataset Preview:")
print(df.head())
```

```
# Convert Gender column into numeric values
le = LabelEncoder()
df["Gender"] = le.fit_transform(df["Gender"])
```

```
# Features and Target
X = df[["Gender", "Age", "EstimatedSalary"]]
y = df["Purchased"]
```

```
# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
```

```
X, y, test_size=0.3, random_state=42
)

# Feature Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create Logistic Regression Model
model = LogisticRegression(max_iter=1000)

# Train Model
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
print("\nAccuracy:", accuracy_score(y_test, y_pred))

# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Plot Confusion Matrix
plt.figure(figsize=(6,4))
sns.heatmap(cm,
            annot=True,
            fmt="d",
            cmap="Blues")

plt.title("Logistic Regression Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Linear Regression

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Load Dataset
df = pd.read_csv("StudentsPerformance.csv")

# Convert categorical columns into numeric
le = LabelEncoder()

df["gender"] = le.fit_transform(df["gender"])
df["race/ethnicity"] = le.fit_transform(df["race/ethnicity"])
df["parental level of education"] = le.fit_transform(df["parental level of education"])
df["lunch"] = le.fit_transform(df["lunch"])
df["test preparation course"] = le.fit_transform(df["test preparation course"])

# Features and Target
X = df[[
    "gender",
    "race/ethnicity",
    "parental level of education",
    "lunch",
    "test preparation course",
    "reading score",
    "writing score"
]]

y = df["math score"]

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create Model
model = LinearRegression()

# Train Model
model.fit(X_train, y_train)
```

```

# Predict
y_pred = model.predict(X_test)

# Evaluation
print("R2 Score:", r2_score(y_test, y_pred))
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))

# Graph
plt.figure(figsize=(8,5))
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Math Score")
plt.ylabel("Predicted Math Score")
plt.title("Linear Regression: Actual vs Predicted")
plt.show()

```

K-MEANS

```

import pandas as pd
from sklearn.preprocessing import LabelEncoder

df = pd.read_csv("weather.csv")

for col in df.columns:
    df[col] = LabelEncoder().fit_transform(df[col].astype(str))

print(df.head())
print(df.dtypes)

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Load dataset
df = pd.read_csv("weather.csv")

# Encode all columns
for col in df.columns:
    df[col] = LabelEncoder().fit_transform(df[col].astype(str))

```



```

# KMeans
X = df

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans.fit(X)

# Add cluster labels
df["Cluster"] = kmeans.labels_

print("Clustered Dataset:")
print(df.head())

print("\nCluster Centers:")
print(kmeans.cluster_centers_)

score = silhouette_score(X, kmeans.labels_)
print("\nSilhouette Score:", score)

# Graph
plt.figure(figsize=(8,6))

sns.scatterplot(
    x=df["WindGustDir"],
    y=df["WindDir9am"],
    hue=df["Cluster"],
    s=100
)

plt.title("K-Means Clustering")
plt.xlabel("WindGustDir")
plt.ylabel("WindDir9am")

plt.show()

```

Small code for kmean

```

# Import Libraries
import pandas as pd, matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
from sklearn.cluster import KMeans
# Load Dataset
df = pd.read_csv("weather.csv") # Change dataset name
# Convert categorical columns
for col in df.select_dtypes(include='object').columns:
    df[col] = LabelEncoder().fit_transform(df[col])

```

```
# Select Features
X = df.iloc[:,0:2]      # Select columns for clustering
# K-Means Model
kmeans = KMeans(n_clusters=3, random_state=42)
df["Cluster"] = kmeans.fit_predict(X)
# Plot Clusters
plt.scatter(X.iloc[:,0], X.iloc[:,1], c=df["Cluster"])
plt.scatter(
    kmeans.cluster_centers_[:,0],
    kmeans.cluster_centers_[:,1],
    marker='x',
    s=200
)
plt.title("K-Means Clustering")
plt.show()
```